

Organize o contexto. Preserve a atenção. Prove a execução.

Da oficina ao harness: curadoria prática, interoperável e auditável.

```
ai-context/  
├─ agent/  
├─ playbooks/  
├─ skills/  
├─ knowledge/  
├─ rules/  
├─ contracts/  
├─ evals/  
└─ evidence/
```



Sumário

1	Como ler este E-book	2
2	Introdução - o problema não é falta de documentação	3
3	Parte I - A oficina desorganizada	4
3.1	O manual que parecia completo	4
3.2	O custo do manual monolítico	6
3.3	O cartão do chefe da oficina	6
3.3.1	O que não deve entrar no cartão	6
3.4	A ordem de serviço	7
3.4.1	Pré-condições	7
3.4.2	Procedimento	7
3.4.3	Condições de parada	7
3.5	A capacidade especializada	8
3.6	O manual consultivo	8
3.7	O teste e o laudo	8
3.8	Uma jornada completa pela oficina	9
3.9	O mesmo pedido em uma oficina desorganizada	11
3.10	Cinco falhas que a separação precisa evitar	11
3.11	Como saber se a oficina está realmente organizada	12
3.12	Como evoluir o acervo sem voltar ao caos	12
3.13	Os limites da metáfora	13
3.14	O modelo mental completo da oficina	13
4	Parte II - A ponte para engenharia de contexto	14
4.1	Da oficina para o repositório	14
4.2	Contexto não é apenas prompt	14
4.3	Quando carregar: HOT, WARM e COLD	15
4.3.1	HOT	15
4.3.2	WARM	15
4.3.3	COLD	15
4.4	Progressive disclosure	16
4.4.1	Tokens, latência e cache	16
4.5	A árvore mínima	17

5	Parte III - Estrutura técnica para desenvolvimento	18
5.1	O arquivo backend caótico	18
5.2	Agent: o conteúdo HOT	18
5.3	Playbook: coordenação WARM	19
5.4	Skill: capacidade ativável	20
5.5	Knowledge: fundamentação WARM ou COLD	21
5.6	Anatomia dos pacotes	22
5.7	YAML frontmatter: descoberta, roteamento e governança	23
5.7.1	id e name resolvem problemas diferentes	23
5.7.2	Frontmatter de uma skill	24
5.7.3	Frontmatter de um playbook	25
5.7.4	Frontmatter de uma rule	25
5.7.5	Para curiosos - anatomia ampliada de uma skill	25
5.8	Rules: obrigação, proibição e fallback	29
5.9	Contracts: o encaixe verificável	30
5.10	Evals: verificar recuperação e aplicação	30
5.11	Evidence Records: tornar a execução auditável	31
5.11.1	Quando usar	32
5.11.2	Exercício copiável: auditar uma execução	32
5.11.3	Massa mínima para experimentar	33
5.11.4	Exemplo de saída	33
5.11.5	Como interpretar o resultado	34
5.11.6	Diagnóstico por camada	34
5.12	Reduzindo omissão e alucinação operacional	35
6	Parte IV - Estudo de caso: operação assíncrona	36
6.1	O requisito	36
6.2	O fluxo confiável	36
6.3	Contrato de aceitação	37
6.4	Contrato de progresso e conclusão	37
6.5	Estados e transições	38
6.6	Edge cases obrigatórios	38
6.7	Massa progressiva de testes	38
6.8	Evidência esperada	39
7	Parte V - Governança e operação do contexto	40
7.1	Classificação híbrida	40
7.2	Fonte de autoridade e resolução de conflitos	41
7.3	Metadados mínimos	41
7.4	Convenções de escrita e delimitadores	41
7.4.1	Convenção recomendada	42
7.5	Orçamento de contexto	46
7.6	Ciclo de vida dos artefatos	46
7.7	Refatoração orientada por recuperabilidade	46
7.8	Pequeno loop de implementação e testes	47
7.9	Adoção progressiva	47

7.10	Antipadrões recorrentes	47
7.11	Crítérios de sucesso	48
8	Parte VI - Templates práticos	49
8.1	Template de Agent Card	49
8.2	Template mínimo de Skill	49
8.3	Template de Playbook	50
8.4	Template de Rule	51
8.5	Template de Contract	51
8.6	Template de Eval	51
8.7	Template de Evidence Record	52
8.8	Checklist do curador	52
8.8.1	Estrutura	52
8.8.2	Recuperação	52
8.8.3	Aplicação	52
8.8.4	Operação	53
9	Glossário	54
10	Conclusão	56
11	Referências e leituras recomendadas	57
11.1	Especificações e guias	57
11.2	Fundamentos de aprendizagem e contexto	57
11.3	Tecnologias do estudo de caso	57

i Propósito desta edição

Esta edição preserva a progressão em três movimentos - oficina, ponte conceitual e backend - e acrescenta um contrato explícito para identidade, descoberta e governança dos artefatos. A anatomia ampliada de skills aparece como aprofundamento opcional, sem ser confundida com requisito da especificação Agent Skills.

! Exemplo mecânico fictício

O motor, os procedimentos, as medições e as especificações deste livro são exclusivamente didáticos. Não utilize o conteúdo para manutenção mecânica real.

! Limites de sistemas com IA

Modelos de linguagem possuem comportamento probabilístico. Organização de contexto, skills, regras, contratos e avaliações reduzem riscos, mas não garantem resultados perfeitos.

Capítulo 1

Como ler este E-book

O livro foi desenhado para desenvolvedores que conhecem software, mas estão entrando no universo de agentes, context engineering e sistemas orientados por LLM.

Existem duas rotas:

Rota essencial. Leia o fluxo principal. Ele apresenta a oficina, realiza a ponte para backend e termina com um exemplo assíncrono completo.

Rota de aprofundamento. Leia também as caixas “Para curiosos” e “Deep Dive”. Elas tratam de classificação híbrida, governança, conflitos, avaliação entre modelos e orçamento de contexto.

Os marcadores possuem funções estáveis:

Marcador	Função
Exemplo	Demonstra uma aplicação reduzida.
Cuidado	Expõe risco ou interpretação incorreta.
Dica	Resume uma decisão prática.
Evidência	Mostra como provar uma conclusão.
Para curiosos	Aprofundamento opcional.

O fluxo principal contém o necessário para compreender e aplicar o método. Conteúdo normativo, stop conditions e critérios de evidência nunca são escondidos em aprofundamentos opcionais.

Capítulo 2

Introdução - o problema não é falta de documentação

Times de desenvolvimento acumulam instruções em README, arquivos de agentes, wikis, comentários, tickets, ADRs, contratos, exemplos e prompts. Quando um agente de IA recebe uma tarefa, essa informação precisa ser descoberta, selecionada, carregada, interpretada e aplicada.

Adicionar mais conteúdo não resolve automaticamente o problema. Um documento pode ser completo e ainda falhar porque:

- mistura obrigação com explicação;
- esconde regras em exemplos;
- não informa quando um procedimento deve ser ativado;
- possui fontes divergentes;
- carrega detalhes irrelevantes em todas as tarefas;
- não permite provar qual informação foi utilizada;
- não possui testes de recuperabilidade.

Context engineering é o trabalho de selecionar e manter o conjunto de informações que aumenta a chance do comportamento desejado. O objetivo não é o menor prompt possível. É o **contexto mínimo suficiente** para decidir, executar e verificar.



Minimal não significa curto

Um agente com poucas linhas pode ser vago. Um agente um pouco maior pode ser mais eficiente se concentrar responsabilidade, invariantes, roteamento, critérios de parada e evidência final.

Esta edição começa longe do backend de propósito. Antes de olhar pastas, schemas ou APIs, vamos observar uma oficina em que todo conhecimento foi colocado no mesmo manual.

Capítulo 3

Parte I - A oficina desorganizada

3.1 O manual que parecia completo

Uma oficina decidiu registrar em um único documento tudo que seus mecânicos precisavam saber sobre montagem e diagnóstico de motores.

O manual continha:

- a responsabilidade do mecânico;
- explicações sobre peças;
- procedimentos de desmontagem e montagem;
- regras de segurança;
- valores de referência;
- histórias de serviços antigos;
- formulários de inspeção;
- testes finais;
- dicas transmitidas por profissionais experientes.

Em uma primeira leitura, a solução parecia excelente. Tudo estava no mesmo lugar. Ninguém precisava procurar em várias fontes.

O problema surgiu durante um serviço urgente. O mecânico precisava descobrir rapidamente:

1. qual motor estava diante dele;
2. qual instrução era obrigatória;
3. qual procedimento se aplicava àquele modelo;
4. quando deveria interromper;
5. como comprovar que o serviço havia terminado corretamente.

O manual apresentava todas essas informações, mas elas estavam misturadas. Uma observação histórica aparecia ao lado de uma regra de segurança. Uma dica informal tinha o mesmo destaque de uma especificação aprovada. Um procedimento de outro modelo parecia semelhante o suficiente para induzir uma decisão incorreta.

O documento fictício abaixo é deliberadamente ruim. Leia como se você precisasse executar um serviço urgente e observe quantas responsabilidades precisa separar mentalmente.

<!-- ANTI-PATTERN: responsabilidades misturadas no mesmo arquivo -->

Você é um mecânico sênior responsável por montar e diagnosticar motores.

O cabeçote fecha a parte superior do motor e ajuda a formar a câmara de combustão. Nunca invente valores de torque e não continue se o modelo exato do motor não estiver identificado.

Para instalar o cabeçote, limpe as superfícies, verifique a planicidade, confirme a junta correta, consulte a sequência de aperto, aplique a especificação correspondente e registre o resultado.

Para verificar a planicidade, posicione a régua de precisão nos sentidos longitudinal, transversal e diagonal. Utilize o instrumento apropriado e compare o maior desvio com a fonte aprovada.

O motor fictício Aurora X1 utiliza a tabela TQ-AUR-X1. Outros motores podem utilizar materiais, parafusos e sequências diferentes. Nunca reutilize um parafuso identificado como de uso único.

Se o motor apresentar baixa compressão depois da montagem, verifique vedação, válvulas, junta, sincronismo e condições do cilindro.

Use o formulário de montagem para registrar o motor, as peças, o instrumento utilizado, a sequência e o responsável pela inspeção.

Os primeiros cabeçotes eram frequentemente construídos com materiais diferentes dos utilizados em motores modernos. Mudanças de material podem alterar expansão térmica e métodos de inspeção.

Depois da montagem, execute os testes de vedação e compressão. Não considere o serviço concluído apenas porque o motor ligou.

Se o resultado estiver fora da especificação, não tente compensar aumentando o torque. Interrompa e execute o diagnóstico.

Todas as frases podem ser úteis, mas não possuem a mesma função.

Trecho	Função percebida
"Você é um mecânico sênior..."	Define papel.
"O cabeçote fecha..."	Explica o domínio.
"Nunca invente valores..."	Impõe uma regra.
"Para instalar o cabeçote..."	Coordena um procedimento.
"Para verificar a planicidade..."	Descreve capacidade especializada.
"Os primeiros cabeçotes..."	Registra contexto histórico.

Trecho	Função percebida
“Use o formulário...”	Indica recurso de saída.
“Não considere concluído...”	Define teste e critério de conclusão.

O primeiro aprendizado é simples: **proximidade textual não significa responsabilidade igual**.

3.2 O custo do manual monolítico

O manual único cria quatro dificuldades:

- **Decisão:** o mecânico precisa ler explicações extensas antes de encontrar o próximo passo.
- **Autoridade:** uma história antiga pode parecer tão obrigatória quanto uma regra atual.
- **Aplicabilidade:** um procedimento correto para determinado motor pode ser perigoso em outro.
- **Evidência:** mesmo quando o serviço funciona, outra pessoa pode não conseguir reconstruir qual fonte, medição ou decisão foi utilizada.



Conteúdo completo pode continuar inutilizável

O problema não é apenas ausência de conhecimento. É dificuldade de localizar a informação correta, reconhecer sua autoridade e aplicá-la na situação adequada.

3.3 O cartão do chefe da oficina

O primeiro refactor não cria várias pastas. Ele separa o que o mecânico precisa lembrar em praticamente todos os serviços.

O cartão do chefe contém:

- responsabilidade: coordenar montagem e diagnóstico;
- identificação: confirmar motor e componente antes de agir;
- segurança: não inventar especificações;
- encaminhamento: usar a ordem de serviço correspondente;
- especialização: chamar quem possui uma habilidade específica;
- parada: interromper quando fonte, ferramenta ou condição estiver ausente;
- conclusão: finalizar somente depois de testes e registros.

O cartão é curto porque não tenta ensinar mecânica inteira. Sua densidade vem das decisões que concentra.

3.3.1 O que não deve entrar no cartão

Não pertencem ali:

- catálogo completo de motores;
- todas as sequências de montagem;
- histórico de incidentes;
- formulários de todas as inspeções;
- explicações extensas sobre materiais;
- medições específicas de um componente.

Se todos os detalhes voltarem para o cartão, o refactor apenas mudará o nome do manual monolítico.

3.4 A ordem de serviço

O cartão informa que um procedimento precisa ser escolhido. A ordem de serviço coordena uma atividade específica.

Para instalar um cabeçote, uma ordem de serviço poderia conter:

3.4.1 Pré-condições

- motor identificado;
- peça compatível;
- manual aplicável disponível;
- instrumentos verificados;
- superfície preparada.

3.4.2 Procedimento

1. Inspecionar as superfícies.
2. Medir a planicidade.
3. Validar junta e parafusos.
4. Consultar sequência e valores aplicáveis.
5. Executar a instalação.
6. Registrar medições e componentes.
7. Realizar os testes finais.

3.4.3 Condições de parada

- especificação ausente;
- medição fora da tolerância;
- componente incompatível;
- instrumento sem verificação;
- evidência insuficiente.

A ordem de serviço não precisa explicar profundamente a função de cada componente. Ela coordena pessoas, fontes, instrumentos e verificações para atingir um resultado.

3.5 A capacidade especializada

Durante a instalação, medir planicidade exige habilidade, instrumento e método próprios. Essa capacidade pode ser usada em diferentes serviços: diagnóstico, montagem, retífica ou inspeção.

Uma capacidade especializada possui:

- condição clara de ativação;
- escopo delimitado;
- método conhecido;
- entradas necessárias;
- resultado verificável;
- critérios de falha;
- registro correspondente.

Nem todo procedimento é uma capacidade independente. “Instalar o cabeçote” coordena várias ações e fontes. “Medir planicidade” é uma capacidade reutilizável que pode participar de diferentes ordens de serviço.



Coordenação e capacidade resolvem problemas diferentes

A ordem de serviço responde “como atingir este objetivo?”. A capacidade especializada responde “como executar bem esta atividade reutilizável?”.

3.6 O manual consultivo

O manual deixa de ser uma mistura de tudo e passa a fundamentar decisões.

Ele pode explicar:

- função dos componentes;
- diferenças entre materiais;
- tipos de falha;
- instrumentos de medição;
- sinais de desgaste;
- compatibilidade entre peças;
- histórico de determinado modelo.

O manual não coordena sozinho o serviço. Também não deve esconder uma obrigação crítica em um parágrafo explicativo.

Quando uma regra muda, ela precisa possuir fonte e autoridade reconhecíveis. Quando uma explicação muda, o impacto pode ser diferente. Separar essas responsabilidades reduz o risco de tratar conhecimento ilustrativo como determinação obrigatória.

3.7 O teste e o laudo

Uma oficina organizada não considera o trabalho concluído apenas porque o motor ligou.

O teste responde se o resultado atende aos critérios esperados. O laudo registra:

- identificação do motor;
- ordem de serviço utilizada;
- fonte consultada;
- instrumentos e medições;
- peças aplicadas;
- testes executados;
- resultado;
- exceções e riscos residuais.

Encontrar a instrução correta não prova que ela foi aplicada. Aplicar o procedimento não prova que o resultado foi validado. Validar o resultado não prova que outra pessoa conseguirá reconstruir a execução.

A sequência passa por cinco momentos: escolher a ordem correta, consultar o manual aplicável, executar a capacidade necessária, realizar o teste e registrar o laudo.



O laudo fecha o ciclo

Outra pessoa deve conseguir compreender o que foi feito, quais fontes foram utilizadas, quais testes passaram e por que o serviço foi considerado concluído.

3.8 Uma jornada completa pela oficina

Para testar o modelo mental, acompanhe um serviço fictício do início ao fim.

Um veículo chega com perda de potência depois de um reparo recente. O cliente relata aquecimento e consumo de líquido de arrefecimento. A oficina não sabe ainda se a causa está no cabeçote, na vedação, na circulação ou em outro componente.

1. Recepção e identificação.

A primeira atividade não é desmontar. A oficina registra:

- identificação do veículo e do motor;
- serviço anterior conhecido;
- sintomas relatados;
- momento em que a falha aparece;
- alertas observados;
- autorização inicial.

Se a identificação do motor estiver incompleta, o cartão do chefe obriga a interromper a seleção do procedimento. Uma ordem aparentemente semelhante não pode ser escolhida por conveniência.

2. Escolha da ordem de serviço.

O chefe seleciona uma ordem de diagnóstico inicial, não uma ordem de instalação. Essa diferença importa: o sintoma ainda não prova a causa.

A ordem escolhida orienta:

1. confirmar os sintomas;
2. executar inspeções não invasivas;
3. registrar medições iniciais;
4. comparar resultados com a fonte aplicável;
5. decidir se existe fundamento para desmontagem;
6. solicitar autorização adicional quando necessária.

Uma oficina desorganizada pode pular diretamente para “trocar a junta”. A oficina organizada distingue hipótese, teste e conclusão.

3. Consulta ao manual.

O mecânico consulta apenas as seções relacionadas ao motor identificado e aos testes previstos. O manual explica o significado das medições, os possíveis padrões de falha e as limitações de cada teste.

Uma medição isolada pode ter várias interpretações. A função do manual é sustentar a leitura; a função da ordem de serviço é dizer quando essa leitura participa da decisão.

4. Ativação de capacidades especializadas.

A ordem exige duas capacidades que não estão disponíveis com o mesmo profissional:

- teste do sistema de arrefecimento;
- medição de vedação dos cilindros.

Cada especialista recebe a entrada necessária, executa o método conhecido e devolve um resultado verificável. O chefe não precisa copiar o método completo para dentro da ordem de serviço.

5. Decisão intermediária.

Os resultados apontam para perda de vedação, mas ainda existe uma inconsistência entre duas medições. A condição de parada impede a desmontagem automática.

A oficina então:

- repete a medição com instrumento verificado;
- registra o valor anterior e o novo;
- consulta a nota aplicável ao modelo;
- confirma a hipótese somente depois da convergência.

Parar não representa fracasso. É uma decisão de segurança quando o fundamento ainda é insuficiente.

6. Execução e conclusão.

Depois da autorização, outra ordem coordena a desmontagem, inspeção, preparação, montagem e testes finais. As capacidades especializadas são acionadas nos momentos correspondentes.

O veículo não é liberado porque o motor simplesmente entrou em funcionamento. A oficina verifica os critérios definidos e produz um laudo com medições, componentes, fontes e resultado.

★ A jornada preserva a diferença entre funções

O cartão orienta decisões permanentes; a ordem coordena o objetivo; as capacidades executam atividades delimitadas; o manual explica; o teste verifica; o laudo registra.

3.9 O mesmo pedido em uma oficina desorganizada

Agora imagine o mesmo serviço com o manual monolítico.

O primeiro mecânico encontra um relato antigo sobre um motor semelhante e conclui que a junta é a causa mais provável. O segundo lembra de uma recomendação informal e sugere outro teste. O chefe possui uma atualização mais recente, mas ela está em uma folha separada. O especialista executa uma medição correta, porém não registra a condição do instrumento. O motor funciona ao final, mas ninguém consegue explicar por que determinada sequência foi escolhida.

O problema não foi falta de experiência. Cada pessoa tinha uma parte útil do conhecimento. O fracasso surgiu porque a oficina não conseguia responder com segurança:

- qual informação tinha força obrigatória;
- qual informação era apenas histórica;
- qual procedimento se aplicava;
- qual especialista deveria participar;
- qual resultado permitia avançar;
- qual registro provava a conclusão.

3.10 Cinco falhas que a separação precisa evitar

- **Instrução órfã:** uma folha diz “verifique a vedação”, mas não informa em qual serviço, para qual motor ou com qual resultado esperado. Conecte a instrução a uma ordem, a uma fonte aplicável e a um registro.
- **Ordem que tenta ensinar tudo:** a ordem copia capítulos inteiros do manual e descreve todos os instrumentos. Mantenha a coordenação na ordem e consulte o detalhe quando a etapa exigir.
- **Especialista sem limite:** o profissional domina uma medição, mas passa a decidir sozinho se o veículo pode ser liberado. Delimite a capacidade e devolva o resultado à coordenação responsável.
- **Dica tratada como obrigação:** uma solução antiga passa a ser aplicada como regra geral. Verifique origem, aplicabilidade e autoridade antes de transformar experiência em determinação.
- **Laudo sem lastro:** o documento declara “aprovado” sem relacionar testes, medições e critérios. Registre evidências que permitam conferir a conclusão.

3.11 Como saber se a oficina está realmente organizada

Uma árvore de armários bem etiquetada não basta. A organização precisa melhorar o trabalho observado.

Perguntas práticas:

- O mecânico identifica rapidamente a ordem aplicável?
- Uma busca por um sintoma conduz à fonte correta?
- Ordens inadequadas permanecem fora da seleção?
- O profissional reconhece quando deve parar?
- Capacidades especializadas devolvem resultados comparáveis?
- O teste final distingue “motor ligado” de “serviço aprovado”?
- O laudo permite auditoria por outra pessoa?



Organização deve mudar resultados

Se os armários ficaram mais bonitos, mas as pessoas continuam escolhendo instruções erradas, a oficina apenas reorganizou o caos.

3.12 Como evoluir o acervo sem voltar ao caos

Cada novo conteúdo precisa responder a uma pergunta principal.

Pergunta	Destino mais provável
Muda uma decisão permanente do responsável?	Cartão do chefe.
Coordena uma sequência para um objetivo?	Ordem de serviço.
Ensina uma atividade reutilizável?	Capacidade especializada.
Explica o domínio ou sustenta comparação?	Manual consultivo.
Define como verificar ou registrar?	Teste e laudo.

Quando uma informação parece pertencer a todos os lugares, ela provavelmente contém mais de uma responsabilidade e precisa ser dividida.

Uma revisão periódica deve procurar:

- ordens sem responsável;
- capacidades nunca utilizadas;
- manuais sem fonte;
- versões concorrentes;
- registros que não provam nada;
- conteúdo duplicado;
- instruções que não informam quando se aplicam.

3.13 Os limites da metáfora

A oficina ajuda a enxergar funções, mas não substitui arquitetura de software.

No mundo técnico:

- o responsável pode ser um modelo probabilístico;
- a seleção pode depender de busca semântica;
- uma capacidade pode executar código ou chamar uma API;
- a mesma tarefa pode atravessar vários serviços;
- o resultado pode chegar depois de minutos;
- falhas precisam ser correlacionadas automaticamente;
- acesso, privacidade e custo precisam ser medidos.

Essas diferenças justificam a próxima parte. A metáfora construiu o mapa; agora precisaremos de nomes, estruturas e critérios próprios da engenharia de contexto.

3.14 O modelo mental completo da oficina

Antes da transição, a oficina possui cinco elementos:

Elemento	Responsabilidade
Cartão do chefe	Papel, decisões permanentes, encaminhamento e parada.
Ordem de serviço	Sequência coordenada para um objetivo.
Capacidade especializada	Atividade reutilizável e verificável.
Manual consultivo	Conhecimento para compreender e decidir.
Teste e laudo	Verificação e registro do resultado.

Até aqui, não foi necessário discutir repositórios, APIs, schemas ou modelos de linguagem. O objetivo foi construir uma estrutura que o leitor possa reconhecer antes de aprender os nomes técnicos.

Capítulo 4

Parte II - A ponte para engenharia de contexto

4.1 Da oficina para o repositório

Agora os cinco elementos recebem nomes técnicos.

Oficina	Engenharia de contexto	Responsabilidade
Cartão do chefe	Agent	Define papel, invariantes, roteamento, parada e conclusão.
Ordem de serviço	Playbook	Coordena etapas para atingir um objetivo.
Capacidade especializada	Skill	Encapsula capacidade reutilizável e ativável.
Manual consultivo	Knowledge	Fundamenta compreensão e decisão.
Teste e laudo	Eval e Evidence	Verifica comportamento e registra execução.

O mapeamento não afirma que uma oficina funciona como software. Ele destaca uma estrutura comum: coordenação, capacidade, conhecimento, verificação e evidência.

Pesquisas sobre aprendizagem por analogia mostram que comparar explicitamente dois casos pode ajudar o aprendiz a perceber relações comuns, em vez de se prender a detalhes superficiais.¹

4.2 Contexto não é apenas prompt

Em um sistema com agentes, contexto inclui tudo que o modelo pode utilizar durante uma decisão:

¹A separação entre metáfora e aplicação técnica segue o princípio didático de construir correspondências explícitas antes da transferência para um novo domínio.

- instruções do sistema;
- conversa;
- descrição de skills e tools;
- arquivos recuperados;
- resultados de APIs;
- memória;
- plano atual;
- estado da execução;
- mensagens de erro;
- evidências já produzidas.

Context engineering é a disciplina de selecionar, organizar e manter esse estado para aumentar a chance de comportamento correto.²

O objetivo é maximizar utilidade, não simplesmente diminuir texto. Conteúdo irrelevante aumenta ruído. Conteúdo obrigatório ausente produz decisões sem fundamento.

4.3 Quando carregar: HOT, WARM e COLD

Na oficina, alguns elementos permaneciam sempre disponíveis e outros eram consultados somente quando necessários. HOT/WARM/COLD dá nome a essa estratégia de carregamento.

4.3.1 HOT

Conteúdo necessário com frequência para decidir o próximo passo:

- papel do agente;
- invariantes críticas;
- mapa de capacidades;
- condições de parada;
- critérios de conclusão.

4.3.2 WARM

Conteúdo carregado quando um gatilho aplicável é reconhecido:

- playbook de operação assíncrona;
- skill de validação de contrato;
- documentação de uma integração;
- catálogo de falhas de determinado componente.

4.3.3 COLD

Conteúdo raro, histórico ou de aprofundamento:

- decisões antigas;

²A noção de contexto como recurso limitado e selecionado é consistente com práticas contemporâneas de context engineering para agentes.

- comparações extensas;
- casos excepcionais;
- evidências arquivadas;
- estudos detalhados.

Temperatura depende do consumidor. Um catálogo de falhas pode ser WARM para um agente de diagnóstico e COLD para um agente que apenas corrige documentação.



Temperatura não é propriedade física do arquivo

HOT/WARM/COLD descreve uma política de carregamento para determinado consumidor e tarefa. A classificação deve ser reavaliada quando o uso mudar.

4.4 Progressive disclosure

Progressive disclosure entrega informação em camadas.

Catálogo de capacidades

- instrução central ativada
- referência específica recuperada
- script ou tool executada
- evidência produzida

Na especificação Agent Skills, o agente pode receber inicialmente metadados, carregar o SKILL.md quando a skill é ativada e acessar recursos adicionais conforme a necessidade.³

HOT/WARM/COLD não faz parte obrigatória dessa especificação. É uma taxonomia didática deste livro para discutir carregamento.

4.4.1 Tokens, latência e cache

Uma estrutura progressiva pode reduzir tokens quando evita overfetch e duplicação. Ela também pode aumentar o número de leituras, chamadas e decisões de roteamento.

Cache, latência e custo são efeitos do sistema:

Custo por tarefa válida

- = modelo
- + recuperação
- + tools
- + infraestrutura
- + latência operacional
- + retrabalho humano

³A anatomia SKILL.md, scripts/, references/ e assets/, os campos obrigatórios name e description e as restrições de nomenclatura seguem a especificação pública Agent Skills; neste E-book, a mesma anatomia também é usada como convenção organizacional para pacotes que não são skills.

! Segregação não garante economia

Dividir arquivos somente melhora eficiência quando o roteamento encontra a fonte correta, o conteúdo recuperado é suficiente e o número de leituras não destrói o ganho obtido.

Estudos sobre contexto longo mostram que modelos podem utilizar informação de maneira desigual conforme posição e volume, reforçando a necessidade de selecionar e testar o contexto em vez de apenas aumentar a janela.⁴

4.5 A árvore mínima

Depois da ponte, uma estrutura inicial pode ser pequena:

```
ai-context/  
├─ agent/  
├─ playbooks/  
├─ skills/  
└─ knowledge/
```

Essas pastas representam responsabilidades, não uma obrigação universal. Rules, contracts, evals e evidence serão introduzidos somente quando o cenário backend exigir validação formal.

⁴Pesquisas sobre contexto longo mostram que disponibilidade no prompt não garante uso uniforme da informação, especialmente em posições intermediárias.

Capítulo 5

Parte III - Estrutura técnica para desenvolvimento

5.1 O arquivo backend caótico

Considere um único documento com instruções como:

```
<!-- ANTI-PATTERN: instruções de responsabilidades diferentes no mesmo arquivo -->
```

Você é um desenvolvedor backend sênior.

Use Clean Architecture, DDD e SOLID.

Para operações demoradas, crie jobs, publique em fila e avise o frontend.

O SignalR permite atualização em tempo real.

Nunca notifique antes de persistir o resultado.

POST retorna 202 e GET consulta o status.

Use retry, idempotência, logs e traces.

Rode todos os testes e documente a solução.

O texto mistura:

- papel do agente;
- princípios amplos;
- procedimento assíncrono;
- conhecimento de SignalR;
- contrato HTTP;
- regra transacional;
- política de resiliência;
- critérios de evidência.

Ele parece completo, mas obriga o modelo a inferir quais itens são permanentes, quais dependem da tarefa e quais possuem autoridade normativa.

5.2 Agent: o conteúdo HOT

O agent deve permanecer pequeno e decisório.

```
---
id: AGENT-BACKEND-001
name: Backend Development Agent
type: agent
description: Coordena mudanças backend pequenas, seguras e verificáveis.
temperature: hot
owner: backend-platform
---
```

Backend development agent

Responsabilidade: implementar mudanças pequenas e verificáveis.

- Mudanças HTTP exigem validação de contrato.
- Operações mutáveis exigem política de idempotência.
- Processamento em background exige o playbook assíncrono.
- Ative skills de API, persistência ou mensageria conforme o escopo.
- Não invente dependências, endpoints ou regras ausentes.
- Interrompa quando contrato ou critério de aceite estiver ausente.
- Conclua somente com testes e evidências.

HOT não significa apenas poucas linhas. O conteúdo precisa concentrar:

1. responsabilidade;
2. invariantes;
3. roteamento;
4. stop conditions;
5. evidência final.

Uma frase como “use as skills quando necessário” é curta, mas transfere toda a decisão para inferência do modelo.

5.3 Playbook: coordenação WARM

O playbook é carregado quando a tarefa corresponde ao seu gatilho.

```
---
id: PLAYBOOK-ASYNC-001
name: Operação Assíncrona
type: playbook
description: Coordena operações longas executadas por job e reconciliadas por API.
temperature: warm
triggers:
  - processamento excede o tempo síncrono esperado
  - conclusão precisa atualizar o front-end
owner: backend-platform
---
```

Operação assíncrona

Quando usar

Usar quando a operação não puder concluir dentro do tempo síncrono esperado.

Pré-condições

- Contrato de entrada definido.
- Identidade e autorização disponíveis.
- Política de idempotência definida.
- Estados do job conhecidos.

Fluxo

1. Validar a requisição.
2. Criar o job durável.
3. Publicar a mensagem.
4. Processar com correlação.
5. Persistir o resultado.
6. Confirmar a transação.
7. Notificar o frontend.
8. Preservar reconciliação via API.

Stop conditions

- Contrato ausente.
- Estado final não persistível.
- Resultado da tool não verificável.
- Notificação sem identificador correlacionável.

O playbook coordena. Ele pode ativar skills, consultar knowledge, aplicar rules e validar contracts.



Playbook fora de knowledge

Neste modelo, playbooks/ permanece na raiz porque coordena execução e possui gatilhos próprios. Uma plataforma pode armazená-lo fisicamente sob outro namespace, mas deve preservar o tipo `playbook`, a descoberta e a força operacional. Localização física e responsabilidade semântica não são a mesma decisão.

5.4 Skill: capacidade ativável

Uma skill encapsula uma capacidade especializada reutilizável.

`validate-api-contract/`

└─ SKILL.md

```
|— references/
|   └─ http-contract-guidance.md
|— scripts/
|   └─ validate-openapi.py
└─ assets/
    └─ contract-report-template.yaml
```

name: validate-api-contract

description: >

Use esta skill para analisar ou validar mudanças em contratos HTTP, OpenAPI, status codes, headers e compatibilidade de clientes.

metadata:

display-name: Validar Contrato de API

owner: backend-platform

temperature: warm

authority: advisory

Validar contrato de API

1. Identifique a operação e os consumidores.
2. Compare entrada, saída, erros e headers.
3. Execute o validador quando disponível.
4. Registre incompatibilidades e evidências.

A descrição participa da descoberta. Se for ampla demais, a skill ativa em excesso. Se for estreita demais, pode não ser encontrada.



Playbook não é skill

Um playbook coordena várias capacidades para atingir um objetivo. Uma skill encapsula uma capacidade reutilizável. Não transforme todo conteúdo segregável em skill: uma pasta com `references/`, `scripts/` ou `assets/` continua sendo apenas um pacote até possuir `SKILL.md`, descrição de descoberta e responsabilidade ativável.

5.5 Knowledge: fundamentação WARM ou COLD

Knowledge contém informação usada para compreender, interpretar ou decidir:

- limites do domínio;
- mapa da arquitetura;
- schemas de banco;
- documentação de APIs;
- comportamento de mensageria;
- convenções de observabilidade;
- catálogo de falhas;

- decisões históricas.

Knowledge não deve duplicar integralmente agent ou playbook. O agent roteia. O playbook coordena. A knowledge fundamenta.

```
---
id: KNOWLEDGE-SIGNALR-001
name: SignalR em Operações Assíncronas
type: knowledge
description: Explica o papel do SignalR em operações assíncronas.
temperature: warm
authority: advisory
owner: backend-platform
---
```

SignalR em operações assíncronas

Use para

- compreender atualização em tempo real;
- comparar push e consulta por API;
- reconhecer limites de entrega.

Limite essencial

SignalR reduz a latência percebida, mas não substitui o estado durável.

Fontes

- documentação oficial do ASP.NET Core SignalR;
- contrato interno da operação assíncrona.

Ex Como reconhecer knowledge

Se o conteúdo explica o domínio, compara alternativas ou sustenta uma decisão sem coordenar uma sequência de execução, ele tende a ser knowledge.

5.6 Anatomia dos pacotes

A anatomia inspirada em Agent Skills pode ser reutilizada em outros pacotes sem afirmar que tudo é uma skill.

- **references/ - conteúdo consultado:** schemas, política de retry, documentação de integração, tabelas de compatibilidade e catálogo de erros. O agente lê apenas a referência necessária para fundamentar uma decisão.
- **scripts/ - execução determinística:** validadores de schema, comparação de contratos, linters, geração de relatórios e verificação de links. O agente executa o

script e registra comando, parâmetros, código de saída e resultado.

- **assets/ - recurso de entrada ou saída:** templates, exemplos-base, schemas de relatório, boilerplates e formulários. O agente usa ou copia o recurso; não o trata como instrução normativa.

```
validate-api-contract/
├─ SKILL.md
├─ references/
│   └─ http-contract-guidance.md
├─ scripts/
│   └─ validate-openapi.py
└─ assets/
    └─ contract-report-template.yaml
```

★ Separar leitura de recurso de saída

Reference é lida para orientar a decisão. Script é executado. Asset é consumido ou copiado para produzir a saída.

5.7 YAML frontmatter: descoberta, roteamento e governança

Frontmatter é o bloco YAML delimitado por `---` no topo de um artefato. Ele oferece metadados que podem ser lidos sem carregar todo o corpo.

Use-o para:

- descobrir o artefato pela descrição;
- filtrar por tipo, domínio, estado ou autoridade;
- reconhecer gatilhos antes de carregar o procedimento;
- relacionar rules, contracts, playbooks e evals;
- controlar proprietário, versão e revisão;
- manter metadados estáveis na parte inicial do contexto.

! Frontmatter não garante cache

Metadados estáveis podem favorecer roteamento e reutilização de prefixos quando o runtime e o provedor oferecem prompt caching. A economia depende de serialização estável, posição, política do provedor e frequência de reutilização. Meça o efeito; não o trate como propriedade automática do YAML.

5.7.1 id e name resolvem problemas diferentes

Em artefatos internos, use `id` e `name` juntos quando o item for reutilizável e catalogado:

Campo	Responsabilidade	Regra prática
id	Identidade técnica estável para referências, relacionamentos, auditoria e migração.	Não deve mudar apenas porque o título foi reescrito.
name	Nome legível para catálogos, interfaces, buscas e revisão humana.	Pode evoluir sem quebrar referências pelo id.
description	Explica o que o artefato faz e quando é relevante.	Deve permitir descoberta sem abrir o corpo inteiro.

Agent, playbook, knowledge, rule, contract e eval se beneficiam dessa dupla. Um Evidence Record de execução é diferente: ele é uma instância imutável identificada por `evidenceId`, não um artefato reutilizável descoberto por nome. O schema ou tipo de Evidence Record pode ter `id` e `name`; cada registro produzido não precisa repetir um nome decorativo.



O name de Agent Skills possui contrato próprio

No frontmatter de SKILL.md, `name` é identificador interoperável: deve usar letras minúsculas, números e hífens, possuir no máximo 64 caracteres e coincidir com o nome do diretório. Um título humano em Pascal Case ou Title Case pode aparecer no primeiro # do corpo ou em `metadata.display-name`; não substitua o `name` exigido pela especificação.^a

^aA anatomia SKILL.md, `scripts/`, `references/` e `assets/`, os campos obrigatórios `name` e `description` e as restrições de nomenclatura seguem a especificação pública Agent Skills; neste E-book, a mesma anatomia também é usada como convenção organizacional para pacotes que não são skills.

5.7.2 Frontmatter de uma skill

Na especificação Agent Skills, `name` e `description` são obrigatórios e participam da descoberta. Campos próprios da organização devem permanecer dentro de `metadata`, a menos que o cliente utilizado documente outra extensão.

```
---
name: validate-api-contract
description: >
  Use para validar contratos HTTP, OpenAPI, status codes,
  headers e compatibilidade de clientes.
metadata:
  display-name: Validar Contrato de API
  owner: backend-platform
  temperature: warm
  authority: advisory
---
```

5.7.3 Frontmatter de um playbook

```
---
id: PLAYBOOK-ASYNC-001
name: Operação Assíncrona
type: playbook
description: Coordena operações longas executadas por job.
temperature: warm
triggers:
  - operação excede o tempo síncrono esperado
  - conclusão precisa ser reconciliável
owner: backend-platform
status: approved
version: 2.1.0
---
```

5.7.4 Frontmatter de uma rule

```
---
id: RULE-ASYNC-001
name: Persistir Antes de Notificar
type: rule
description: Exige persistência antes da notificação ao cliente.
temperature: warm
authority: normative
appliesTo:
  - async-jobs
owner: backend-platform
status: approved
version: 2.1.0
---
```

Evite campos voláteis no conteúdo sempre carregado, como timestamp de cada execução. Eles reduzem estabilidade, dificultam comparação e podem invalidar reaproveitamento de prefixos.

5.7.5 Para curiosos - anatomia ampliada de uma skill

Convenção interna, não extensão obrigatória do padrão

A estrutura abaixo é uma proposta de curadoria para equipes que precisam operar skills com regras, perguntas, evidências e conteúdo progressivo. O padrão Agent Skills exige apenas o contrato publicado para SKILL.md; as seções do corpo são livres. Adote a anatomia ampliada somente quando ela melhorar execução e recuperabilidade, e teste a skill no cliente real.

Avaliação da proposta. A ideia original é tecnicamente coerente porque reúne descoberta, escopo, roteamento, guardrails e verificação. Entretanto, quatro correções

evitam transformar a skill em um novo arquivo monolítico:

- sinais de ativação devem descrever situações semânticas e exemplos positivos ou negativos, não um dicionário gigante de palavras;
- regras organizacionais compartilhadas devem ser referenciadas por id e caminho, em vez de copiadas para cada skill;
- perguntas obrigatórias devem seguir **descobrir -> verificar -> perguntar**: a skill não pergunta ao operador o que consegue comprovar no repositório;
- receitas HOT não devem congelar portas, métricas ou limites que mudam por ambiente sem apontar uma fonte normativa vigente.

Frontmatter interoperável e título humano:

```
---
name: kubernetes-manifests
description: >
  Cria ou revisa manifests Kubernetes para APIs, workers e funções.
  Use quando a tarefa mencionar Deployment, Service, HPA, env ou secrets.
compatibility: Requer acesso ao repositório e ao validador YAML aprovado pelo time.
metadata:
  display-name: Kubernetes Manifests
  owner: platform-engineering
  version: "1.0.0"
  temperature: warm
---
```

Kubernetes Manifests

O name em kebab-case identifica a skill. # Kubernetes Manifests é o título de leitura. metadata.display-name é opcional e só deve ser usado quando o catálogo interno realmente o consome.

Corpo recomendado:

- **Quando usar:** delimite tarefas cobertas e resultado esperado.
- **Quando não usar:** exclua tarefas vizinhas que pertencem a outra skill ou exigem aprovação.
- **Entradas e pré-condições:** declare dados, arquivos, acesso e ferramentas necessários.
- **Sinais de ativação:** inclua situações, intenções e dois ou três exemplos; use termos apenas como apoio à descoberta.
- **Fluxo principal:** descreva a sequência curta de inspeção, decisão, execução e validação.
- **Guardrails da camada:** referencie rules NORMATIVE, proibições, permissões e stop conditions.
- **Recursos empacotados:** liste somente diretórios e arquivos que existem.
- **Catálogo progressivo:** informe quando abrir cada referência, sem carregá-la antecipadamente.
- **Receitas compactas:** registre decisões recorrentes em até três linhas e aponte a

fonte detalhada.

- **Avisos obrigatórios de saída:** defina mensagens condicionais que o usuário precisa receber.
- **Perguntas e condições de parada:** pergunte apenas o que não puder ser descoberto ou verificado.
- **Saída e Definition of Done:** especifique arquivos, validações e estado observável de conclusão.
- **Evidência:** indique comandos, relatórios e identificadores que comprovam a execução.
- **Checklist final:** confirme invariantes antes de responder.

Exemplo compacto do corpo:

Quando usar

Use para criar ou revisar manifests de API, worker ou função quando o repositório já possuir contrato de implantação identificável.

Quando não usar

- Não altere ``ingress/**`` sem autorização explícita.
- Não invente namespace, secret provider, portas ou limites de HPA.

Entradas e pré-condições

- Repositório e ambiente-alvo.
- Contrato de portas e variáveis.
- Rule vigente para disponibilidade e autoscaling.

Sinais de ativação

- Pedido para criar Deployment, Service ou HPA.
- Erro de env, secret, namespace ou porta em manifest existente.
- Não ativar para diagnosticar somente código C# sem impacto de deployment.

Guardrails da camada - NORMATIVE

- APLIQUE ``RULE-K8S-NAMING-001`` para derivar identificadores.
- APLIQUE ``RULE-K8S-SECRETS-002``; nunca grave segredo em texto puro.
- PARE se ambiente, namespace ou contrato de portas não puder ser verificado.

Recursos empacotados

Esta skill usa ``SKILL.md`` e ``references/``. Não possui ``scripts/`` nem ``assets/``. Não presuma recursos que não aparecem neste catálogo.

Catálogo progressivo

Tier	Access	Authority	Abrir quando	Path
---	---	---	---	---
HOT	NAVIGATION	NORMATIVE	Após ativar.	<code>`SKILL.md`</code>
WARM	LOOKUP	NORMATIVE	Alterar env ou secret.	<code>`references/env-secrets.md`</code>
WARM	LOOKUP	ADVISORY	Configurar HPA.	<code>`references/hpa.md`</code>
COLD	DEEP DIVE	ILLUSTRATIVE	Investigar legado.	<code>`references/migrations.md`</code>

Receitas compactas - HOT

****HPA:**** use ``autoscaling/v2``; obtenha métrica e alvo em ``RULE-K8S-HPA-001``. Detalhes: WARM ``references/hpa.md``.

****Env e secrets:**** derive nomes da convenção aprovada e valide referências. Detalhes: WARM ``references/env-secrets.md``.

Avisos obrigatórios de saída

- SE um nome for derivado automaticamente, informe origem e transformação.
- SE houver endpoint HTTP, informe se o Ingress ficou fora do escopo.

Perguntas e condições de parada

1. Descubra no repositório se o workload expõe HTTP.
2. Verifique se o tipo é API, worker ou função.
3. Pergunte pelo ambiente somente se não houver evidência confiável.
4. Pare antes de editar quando porta, namespace ou secret provider divergirem.

Saída e Definition of Done

- Manifests alterados somente no escopo autorizado.
- Ordem e referências entre recursos consistentes.
- Validação YAML e policy checks executados com sucesso.
- Nenhum segredo em texto puro.

Evidência e checklist final

- [] Nome derivado do slug e rule citada.
- [] Deployment, Service e HPA coerentes quando aplicáveis.
- [] Portas e probes confirmadas por fonte do repositório.
- [] Conteúdo WARM aberto somente com gatilho registrado.
- [] Comandos, códigos de saída e arquivos alterados registrados.
- [] Avisos condicionais incluídos na resposta.

O catálogo utiliza três eixos separados: Tier indica momento de carga, Access indica forma de descoberta e Authority indica força normativa. Um campo genérico chamado semantic

não deve receber `NORMATIVE`, pois mistura função semântica com autoridade.

! MCP não interpreta frontmatter arbitrário

Tags no YAML podem alimentar um catálogo ou roteador construído pela equipe. O protocolo MCP não transforma automaticamente campos próprios em roteamento. Quando houver integração MCP, alinhe o catálogo interno às descrições e schemas das tools e resources expostos pelo servidor, e valide o comportamento no cliente utilizado.^a

^aMCP define descoberta e chamada de tools e a exposição de resources; a aplicação hospedeira continua responsável por decidir como incorporar, filtrar ou apresentar esses recursos. Campos internos de YAML só produzem efeito quando algum componente da solução os interpreta.

5.8 Rules: obrigação, proibição e fallback

Uma rule explicita comportamento que não deve depender de interpretação informal.

```
---
id: RULE-ASYNC-001
name: Persistir Antes de Notificar
type: rule
description: Exige persistência antes da notificação ao cliente.
temperature: warm
authority: normative
owner: backend-platform
---
```

RULE-ASYNC-001 - Persistir antes de notificar

Obrigação

- DEVE persistir o resultado final.
- DEVE confirmar a transação antes da publicação.

Proibição

- NÃO DEVE tratar SignalR como fonte de verdade.
- NÃO DEVE notificar conclusão antes do commit.

Fallback

- SE a notificação falhar, preserve o estado consultável pela API.

Evidência

- REGISTRE jobId, correlationId, estado persistido e publicação.

Uma regra não deve existir apenas dentro de exemplo histórico ou parágrafo de knowledge.

Como aplicar na prática: extraia a obrigação de uma falha, risco ou política aprovada; declare o comportamento positivo; acrescente a proibição do atalho conhecido; defina fallback e evidência; relacione a rule ao playbook e ao eval que a exercitam.

5.9 Contracts: o encaixe verificável

```
id: CONTRACT-ASYNC-001
name: Iniciar Análise Assíncrona
operation: start-analysis

input:
  required: [investor_id, request_id]

output:
  accepted:
    status: 202
    required: [job_id, status_url]

preconditions:
  - identity_validated
  - idempotency_policy_defined

postconditions:
  - durable_job_exists
  - final_state_is_queryable

forbidden:
  - notify_before_commit
  - finish_without_evidence
```

O contract define entradas, saídas, pré-condições, pós-condições e comportamentos proibidos. Ele reduz ambiguidade entre intenção, execução e teste.

Como aplicar na prática: comece por uma operação observável; modele entrada e saída; transforme critérios de aceite em pós-condições; liste efeitos proibidos; gere fixtures válidas e inválidas; valide o contrato antes de pedir à LLM que proponha a implementação.

5.10 Evals: verificar recuperação e aplicação

Um eval não deve perguntar apenas se o arquivo existe. Ele precisa verificar se o conteúdo correto foi encontrado, interpretado e aplicado na situação adequada.

Uma suíte progressiva pode trabalhar com seis níveis:

Nível	Pergunta	Exemplo de falha
Lexical	Os termos esperados foram encontrados?	A busca não encontra notify-before-commit.
Semântico	Um sinônimo recupera o conteúdo correto?	“Avisar antes de salvar” não encontra a regra.
Contextual	O artefato adequado aparece para o cenário?	Um texto consultivo substitui uma regra obrigatória.
Negativo	Conteúdo irrelevante permanece fora?	Uma operação síncrona recupera o playbook assíncrono.
Aplicação	A orientação foi usada corretamente?	O agente encontra a regra, mas notifica antes do commit.
Ponta a ponta	O fluxo real satisfaz o contrato?	A API retorna 202, mas não cria um job durável.

id: EVAL-RECOVERY-ASYNC-001

name: Recuperar e Aplicar Fluxo Assíncrono

query: "Como informar o front-end quando o job terminar?"

expected:

must_retrieve:

- RULE-ASYNC-001
- CONTRACT-ASYNC-001

must_apply:

- persist_before_notify
- api_status_fallback

must_not_claim:

- signalr_is_source_of_truth

! Recuperação não é conformidade

Encontrar o arquivo correto não prova que a resposta ou a implementação respeitou o conteúdo. Por isso os níveis de aplicação e ponta a ponta são indispensáveis.

Como executar o eval: envie a consulta ao mesmo mecanismo de descoberta usado pelo agente; capture termos, ranking e trechos; execute o agente com esse contexto; aplique asserts determinísticos; use um avaliador semântico apenas no que não puder ser decidido por código; registre a camada exata da falha.

5.11 Evidence Records: tornar a execução auditável

Evidence Record é um registro estruturado que conecta pedido, contexto recuperado, decisão, execução, teste e resultado. Ele não é um log bruto nem uma justificativa literária extensa.

5.11.1 Quando usar

Produza um Evidence Record quando precisar comparar versões do contexto, auditar uma execução, diagnosticar falha de recuperação, avaliar um agente ou comprovar um critério de conclusão.

Antes de executar, separe:

- pedido ou caso avaliado;
- versão do corpus, prompt e modelo;
- fontes que deveriam ser recuperadas;
- regras que deveriam ser aplicadas;
- afirmações e ações proibidas;
- testes determinísticos disponíveis;
- política de mascaramento de dados.

5.11.2 Exercício copiável: auditar uma execução

O prompt abaixo avalia evidências externas. Ele não solicita raciocínio interno da LLM.

ATUE COMO UM AUDITOR DE CONTEXTO.

Objetivo:

Avaliar se a execução fornecida recuperou e aplicou os artefatos corretos.

Entradas:

- CASO: {{pedido_do_usuario}}
- FONTES_ESPERADAS: {{ids_ou_caminhos}}
- FONTES_RECUPERADAS: {{ids_caminhos_e_trechos}}
- REGRAS_ESPERADAS: {{regras_obrigatorias}}
- TOOLS_EXECUTADAS: {{nome_entradas_saidas_status}}
- TESTES_EXECUTADOS: {{ids_e_resultados}}
- RESULTADO_OBSERVADO: {{estado_ou_saida_final}}
- SCHEMA_EVIDENCE_RECORD: {{schema_json}}

Regras de avaliação:

1. Use somente as entradas fornecidas.
2. Não invente arquivo, execução, teste ou resultado ausente.
3. Marque PASS apenas quando fontes, aplicação e resultado forem comprovados.
4. Marque FAIL quando houver violação ou evidência contraditória.
5. Marque INCONCLUSIVE quando faltar evidência para decidir.
6. Diferencie falha de recuperação, aplicação, tool, contrato e resultado.
7. Não exponha raciocínio interno; forneça critérios e evidências observáveis.

Saída obrigatória:

- Evidence Record JSON conforme o schema fornecido.
- Lista curta de evidências ausentes.
- Próximo artefato que deve ser corrigido.

Para executar o exercício:

1. copie o prompt para a LLM aprovada pelo time;
2. substitua cada {{placeholder}} por dados do caso;
3. forneça o schema JSON esperado;
4. grave a resposta junto da versão do corpus;
5. valide campos obrigatórios por código quando possível;
6. compare o diagnóstico com logs, testes e estado persistido.

! LLM-as-a-Judge não é fonte única

O julgamento por LLM é útil para critérios semânticos, mas pode variar entre modelos e execuções. Combine-o com validação de schema, asserts determinísticos e revisão humana amostral.

5.11.3 Massa mínima para experimentar

caso: "Concluir o backtest e atualizar o front-end"

fontesEsperadas:

- RULE-ASYNC-001
- CONTRACT-ASYNC-001
- PLAYBOOK-ASYNC-001

fontesRecuperadas:

- ```
- id: RULE-ASYNC-001
 trecho: "persistir e confirmar antes de notificar"
- id: CONTRACT-ASYNC-001
 trecho: "estado final deve permanecer consultável"
```

regrasEsperadas:

- persist\_before\_notify

toolsExecutadas:

- ```
- name: complete_backtest
  status: success
  correlationId: corr-9812
```

testesExecutados:

- ```
- id: TEST-E2E-ASYNC-007
 result: passed
```

resultado0bservado:

```

durableStatus: completed
signalrEvent: delivered
statusApi: consistent

```

#### 5.11.4 Exemplo de saída

```
{
 "evidenceId": "EV-2026-0042",
 "requestId": "REQ-2026-0188",
 "artifactsRetrieved": [
```

```
"RULE-ASYNC-001",
"CONTRACT-ASYNC-001",
"PLAYBOOK-ASYNC-001"
],
"decision": "persist-before-notify",
"verdict": "PASS",
"tests": [
 { "id": "EVAL-RECOVERY-ASYNC-001", "result": "passed" },
 { "id": "TEST-E2E-ASYNC-007", "result": "passed" }
],
"residualRisks": [],
"timestamp": "2026-08-02T14:30:00Z"
}
```

Um registro útil responde, sem reconstrução manual demorada:

- o que foi solicitado;
- quais fontes sustentaram a decisão;
- qual contrato foi aplicado;
- quais testes foram executados;
- quais resultados foram observados;
- quais riscos permaneceram.

### 5.11.5 Como interpretar o resultado

| Veredito     | Significado                                                           | Próxima ação                                                      |
|--------------|-----------------------------------------------------------------------|-------------------------------------------------------------------|
| PASS         | As evidências comprovam recuperação, aplicação e resultado esperados. | Preservar o registro e monitorar regressões.                      |
| FAIL         | Existe violação ou resultado observável incompatível.                 | Corrigir a camada indicada e repetir apenas os casos afetados.    |
| INCONCLUSIVE | A evidência não permite confirmar nem reprovar.                       | Instrumentar ou coletar o dado ausente antes de alterar o prompt. |

### 5.11.6 Diagnóstico por camada

| Sintoma                                  | Camada provável           | Ajuste inicial                                            |
|------------------------------------------|---------------------------|-----------------------------------------------------------|
| Fonte esperada não apareceu              | Descoberta ou recuperação | Revisar descrição, sinônimos, metadados e índice.         |
| Fonte correta apareceu, mas foi ignorada | Aplicação                 | Tornar obrigação, proibição e prioridade mais explícitas. |
| Tool retornou erro ou dado incompleto    | Execução                  | Revisar contrato, retry, timeout e tratamento de erro.    |

| Sintoma                                 | Camada provável | Ajuste inicial                                                         |
|-----------------------------------------|-----------------|------------------------------------------------------------------------|
| Teste passou, mas estado final divergiu | Ponta a ponta   | Revisar fixture, assert e fonte de verdade.                            |
| Auditor não consegue decidir            | Evidência       | Registrar versão, entradas, fontes, ferramentas e resultados ausentes. |

Um Evidence Record não deve armazenar indiscriminadamente prompts internos, dados pessoais ou payloads completos. Registre IDs, hashes, versões, estados e trechos mínimos necessários conforme a política de retenção.

## 5.12 Reduzindo omissão e alucinação operacional

Estrutura não elimina alucinação. Ela diminui a margem de interpretação e cria pontos verificáveis.

Use quatro elementos complementares:

1. **Obrigação:** o que precisa acontecer.
2. **Proibição:** o que não pode acontecer.
3. **Fallback:** o que fazer quando o caminho principal falha.
4. **Evidência:** como provar que a obrigação foi satisfeita.

*Antes de concluir:*

- [ ] O contrato aplicável foi identificado.
- [ ] As pré-condições foram verificadas.
- [ ] Nenhuma proibição foi violada.
- [ ] O fallback foi preservado.
- [ ] A evidência aponta para resultados observáveis.

**Checklist é barreira, não prova:** ele evita omissões previsíveis; a prova vem de testes, estados persistidos e evidências observáveis.

## Capítulo 6

# Parte IV - Estudo de caso: operação assíncrona

---

### 6.1 O requisito

Uma API inicia um backtest. O processamento pode durar minutos. O front-end precisa exibir o andamento e atualizar a tela quando o trabalho terminar, mesmo que a conexão em tempo real seja temporariamente perdida.

O requisito contém cinco responsabilidades diferentes:

| Responsabilidade                     | Artefato principal         |
|--------------------------------------|----------------------------|
| Aceitar a solicitação                | Contract da API            |
| Coordenar o processamento            | Playbook assíncrono        |
| Executar validações especializadas   | Skills e scripts           |
| Atualizar o cliente conectado        | Canal SignalR              |
| Permitir recuperação após desconexão | Estado durável consultável |

### 6.2 O fluxo confiável

1. Cliente envia POST /backtests.
2. API valida identidade e idempotência.
3. API cria o job durável e retorna 202 + jobId + statusUrl.
4. Worker assume o job e publica progresso persistido.
5. Worker grava o estado final e confirma o commit.
6. Após o commit, o serviço publica a notificação SignalR.
7. Front-end atualiza a tela ao receber o evento.
8. Em reconexão ou dúvida, o front-end consulta statusUrl.

O fluxo possui dois caminhos complementares:

- **push:** reduz a latência percebida e atualiza a interface rapidamente;
- **pull:** recupera a verdade durável quando o evento não chega.

### ! SignalR não é banco de dados

Uma mensagem em tempo real pode ser perdida por desconexão, reinício ou falha transitória. O estado final precisa continuar consultável por API.

## 6.3 Contrato de aceitação

POST /backtests

Idempotency-Key: 2ca9...

HTTP/1.1 202 Accepted

Location: /backtests/jobs/01J...

```
{
 "jobId": "01J...",
 "status": "queued",
 "statusUrl": "/backtests/jobs/01J..."
}
```

Definition of Done da aceitação:

- entrada validada antes da criação do job;
- repetição idempotente não cria trabalho duplicado;
- resposta 202 contém identificador e URL de consulta;
- job existe em armazenamento durável antes da resposta;
- correlationId acompanha toda a operação.

## 6.4 Contrato de progresso e conclusão

```
{
 "eventType": "backtest.completed",
 "jobId": "01J...",
 "version": 7,
 "occurredAt": "2026-08-02T14:30:00Z",
 "status": "completed",
 "resultUrl": "/backtests/jobs/01J.../result"
}
```

Definition of Done da conclusão:

- o estado completed foi persistido antes do evento;
- o evento contém versão monotônica ou mecanismo equivalente;
- o consumidor ignora eventos antigos ou duplicados;
- falha no SignalR não reverte o resultado persistido;
- a API de status devolve a mesma verdade exibida pelo front-end;
- logs e métricas permitem seguir jobId e correlationId.

## 6.5 Estados e transições

```
queued -> running -> completed
 -> failed
 -> cancelled
```

Regras importantes:

- uma transição final não retorna silenciosamente para running;
- cancelamento concorrente possui resultado definido;
- retry não cria uma segunda execução logicamente distinta;
- eventos duplicados não duplicam efeitos no front-end;
- timeout de infraestrutura não é confundido com falha do domínio.

## 6.6 Edge cases obrigatórios

| Caso                               | Resultado esperado                                           |
|------------------------------------|--------------------------------------------------------------|
| Cliente repete o POST              | Mesmo job ou resposta idempotente equivalente.               |
| Worker cai durante o cálculo       | Job volta a ser elegível ou termina em falha controlada.     |
| Commit conclui e SignalR falha     | Status permanece consultável; notificação pode ser repetida. |
| Evento chega duas vezes            | Front-end mantém uma única representação lógica.             |
| Evento antigo chega por último     | Versão mais nova prevalece.                                  |
| Usuário reconecta                  | Tela consulta o status atual e recupera a conclusão.         |
| Usuário tenta consultar job alheio | Acesso negado sem vazamento de existência ou resultado.      |
| Cancelamento disputa com conclusão | Política determinística decide o estado final.               |

## 6.7 Massa progressiva de testes

Organize a massa em quatro camadas progressivas:

- **Camada 1 - contrato:** payload mínimo válido, campos obrigatórios ausentes, formato inválido, identidade sem permissão e chave de idempotência repetida.
- **Camada 2 - máquina de estados:** transições válidas e proibidas, retry após falha transitória, cancelamento concorrente e versão fora de ordem.
- **Camada 3 - integração:** fila, persistência ou hub indisponível, evento duplicado e reinício do worker.
- **Camada 4 - ponta a ponta:** cliente conectado recebe conclusão, cliente desconectado recupera pela API, múltiplas abas convergem e a telemetria correlaciona solicitação, job, commit e publicação.

## 6.8 Evidência esperada

```
operation: backtest-async
jobId: 01J...
contract: CONTRACT-ASYNC-001
stateTransitions:
 - queued
 - running
 - completed
durableState:
 status: completed
 version: 7
notification:
 channel: signalr
 outcome: delivered
fallbackCheck:
 statusApi: consistent
tests:
 passed: 42
 failed: 0
```

Esse registro não precisa conter raciocínio interno do modelo. Ele registra fatos auditáveis e decisões externas relevantes.

## Capítulo 7

# Parte V - Governança e operação do contexto

### 7.1 Classificação híbrida

HOT/WARM/COLD responde a **quando carregar**. Outros eixos respondem a perguntas diferentes. Misturá-los em um único rótulo tende a gerar ambiguidade.

| Eixo        | Pergunta            | Valores possíveis                                                 |
|-------------|---------------------|-------------------------------------------------------------------|
| Temperatura | Quando carregar?    | HOT, WARM, COLD                                                   |
| Acesso      | Como descobrir?     | NAVIGATION, LOOKUP, DEEP DIVE                                     |
| Autoridade  | Quanto obriga?      | NORMATIVE, ADVISORY, ILLUSTRATIVE                                 |
| Tipo        | Qual função cumpre? | Agent, Playbook, Skill, Knowledge, Rule, Contract, Eval, Evidence |

Exemplo:

```
id: RULE-ASYNC-001
name: Persistir Antes de Notificar
temperature: warm
access: lookup
authority: normative
type: rule
```

O modelo híbrido evita conclusões incorretas como “todo conteúdo COLD é pouco importante” ou “todo conteúdo HOT é normativo”. Frequência de carga e força normativa são propriedades independentes.

? Para curiosos - mais eixos

Grandes bases podem acrescentar domínio, sensibilidade, versão, proprietário e prazo de validade. Só adicione um eixo quando ele sustentar recuperação, governança ou auditoria mensurável.

## 7.2 Fonte de autoridade e resolução de conflitos

Quando dois artefatos divergem, o agente não deve escolher pelo texto mais recente no prompt ou pelo arquivo mais detalhado.

Uma política simples pode ordenar:

1. lei, regulação e política corporativa aplicável;
2. contract e rule aprovados;
3. playbook vigente;
4. knowledge consultivo;
5. exemplos ilustrativos.

Em empate de autoridade:

- prefira a versão vigente e aplicável ao domínio;
- verifique proprietário e data de revisão;
- registre o conflito;
- pare e peça decisão quando o impacto for material.

## 7.3 Metadados mínimos

```
id: PLAYBOOK-ASYNC-001
name: Operação Assíncrona com Atualização do Front-end
type: playbook
description: Coordena jobs duráveis com reconciliação por API e notificação.
owner: platform-team
status: approved
version: 2.1.0
reviewedAt: 2026-07-15
reviewAfter: 2026-10-15
appliesTo:
 - dotnet-api
 - background-jobs
related:
 - RULE-ASYNC-001
 - CONTRACT-ASYNC-001
```

Sem proprietário, vigência e relacionamentos, a base pode ficar organizada visualmente e ainda ser operacionalmente insegura.

## 7.4 Convenções de escrita e delimitadores

### 7.4.1 Convenção recomendada

A organização do conteúdo não depende somente de pastas. A forma de escrever ajuda pessoas, parsers e modelos a reconhecer títulos, sequências, exemplos, metadados e limites entre instrução e dado. Essa convenção não é uma linguagem de programação nem uma garantia de obediência: ela reduz ambiguidade quando usada de maneira consistente.

A documentação oficial da Anthropic recomenda instruções claras, listas numeradas quando a ordem importa, exemplos bem delimitados e tags XML para separar tipos de conteúdo em prompts complexos.<sup>1</sup> Essas práticas são úteis além de um único modelo, mas devem ser verificadas nos modelos e clientes realmente usados pelo time.

| Elemento               | Uso recomendado                                                  | Observação                                                   |
|------------------------|------------------------------------------------------------------|--------------------------------------------------------------|
| # Título               | Título único do artefato.                                        | Evite vários títulos de primeiro nível no mesmo arquivo.     |
| ## Seção               | Responsabilidade ou etapa principal.                             | Deve sustentar uma unidade real de leitura.                  |
| ### Subdivisão         | Recorte necessário dentro de uma seção.                          | Não crie subtítulo para uma única frase.                     |
| - item                 | Itens independentes, opções ou propriedades.                     | A ordem não deve alterar o significado.                      |
| 1. etapa               | Sequência obrigatória ou progressiva.                            | Use quando trocar a ordem puder mudar o resultado.           |
| → relação              | Consequência ou relação explicativa simples.                     | É convenção visual, não operador nativo de Markdown.         |
| **rótulo**             | Destacar um rótulo ou decisão curta.                             | Asteriscos não tornam o conteúdo normativo.                  |
| `literal`              | Nome de arquivo, campo, comando ou valor curto.                  | Use uma crase de cada lado para código inline.               |
| Código cercado         | Arquivo, prompt, contrato ou conteúdo literal com várias linhas. | Use três crases e informe a linguagem.                       |
| YAML                   | Frontmatter, metadados e contratos legíveis.                     | YAML exige indentação consistente.                           |
| - [ ] e - [x]          | Checklist pendente e concluído.                                  | O marcador não comprova execução sem evidência.              |
| {{variavel}}           | Placeholder que será substituído.                                | É convenção do projeto; declare quem realiza a substituição. |
| <context>...</context> | Delimitar grupos semânticos em prompts complexos.                | Prefira tags descritivas e sempre feche as tags.             |

<sup>1</sup>A referência da Anthropic recomenda clareza, passos numerados quando a ordem importa, exemplos estruturados e tags XML para separar instruções, contexto e entradas. A formatação deve apoiar a intenção e permanecer consistente; ela não substitui testes de comportamento.

| Elemento                               | Uso recomendado                               | Observação                                                            |
|----------------------------------------|-----------------------------------------------|-----------------------------------------------------------------------|
| <code>&lt;!-- comentário --&gt;</code> | Anotação para autor, parser ou auditoria.     | Pode ficar invisível no HTML, mas continua presente no arquivo bruto. |
| Tabela                                 | Comparação, catálogo ou mapeamento de campos. | Evite colunas demais e texto longo em cada célula.                    |

**Markdown não define autoridade**

Um título grande, negrito ou caixa visual melhora leitura, mas não transforma uma recomendação em rule NORMATIVE. Autoridade precisa estar declarada nos metadados, na fonte e na política de conflitos.

**Títulos, listas, relação e checklist:**

`# Validar contrato de API`

`## Quando usar`

- Alteração de endpoint.
- Mudança de status code ou schema.

`## Fluxo obrigatório`

1. Identificar a operação.
2. Comparar entrada, saída e erros.
3. Executar o validador.
4. Registrar a evidência.

→ O relatório relaciona cada incompatibilidade ao contrato analisado.

`## Checklist`

- `[ ]` Consumidores identificados.
- `[ ]` Compatibilidade avaliada.
- `[x]` Schema sintaticamente válido.

O hífen cria um item sem ordem obrigatória. A numeração informa sequência. O checklist comunica estado editorial, mas somente logs, testes ou Evidence Records demonstram que a ação ocorreu.

**Código inline e código cercado:**

Use uma crase para um trecho curto, como ``jobId``, ``SKILL.md`` ou ``dotnet test``. Para várias linhas, use três crases na abertura e no fechamento. Acrescente a linguagem logo após as crases para separar texto explicativo de conteúdo literal e habilitar realce de sintaxe.

O exemplo abaixo utiliza quatro crases externas apenas para conseguir mostrar,

literalmente, o bloco de três crases que o leitor deverá escrever:

```
```yaml
id: RULE-ASYNC-001
name: Persistir Antes de Notificar
authority: normative
```
```

Linguagens frequentes:

- markdown para artefatos, prompts e exemplos de documentação;
- yaml para frontmatter, configuração e contratos;
- json para payloads, schemas e Evidence Records;
- bash ou powershell para comandos executáveis;
- csharp para exemplos .NET;
- text quando nenhum realce específico for apropriado;
- mermaid somente quando o renderer suportar o diagrama.

### Chaves, placeholders e tags:

Chaves simples fazem parte da sintaxe de objetos JSON, como { "status": "completed" }. Chaves duplas, como {{request\_id}}, não possuem significado universal: neste E-book elas indicam um valor que o operador, template engine ou harness deverá substituir.

Tags como <context>, <instructions>, <input> e <example> se parecem com HTML, mas normalmente são delimitadores XML-like criados para organizar o prompt. Use nomes descritivos, mantenha abertura e fechamento consistentes e não misture dado não confiável dentro de uma tag de instrução.

```
<instructions>
```

```
Analise somente os documentos fornecidos.
```

```
</instructions>
```

```
<context>
```

```
 <rule>Persistir o resultado antes de notificar.</rule>
```

```
 <operation>{{operation_name}}</operation>
```

```
</context>
```

```
<input>
```

```
{{operation_payload}}
```

```
</input>
```



#### Tags não são fronteiras de segurança

Tags ajudam o modelo a reconhecer estrutura, mas não neutralizam prompt injection nem validam a origem do conteúdo. Dados externos continuam exigindo classificação de confiança, delimitação, autorização de tools e validação do resultado.

### Exemplo integrado de um artefato:

```
<!-- ILLUSTRATIVE: exemplo didático -->
```

```

id: PLAYBOOK-ASYNC-001
name: Operação Assíncrona
type: playbook
description: Coordena uma operação durável e notificável.

```

### # Operação Assíncrona

#### ## Quando usar

- A execução excede o tempo síncrono esperado.
- O resultado precisa ser reconciliável pela API.

#### ## Fluxo

1. Validar `{{request_id}}`.
2. Criar o job durável.
3. Processar e persistir o resultado.
4. Confirmar a transação.
5. Notificar o front-end.

→ SignalR reduz latência percebida; a API preserva a fonte de verdade.

#### ## Entrada literal

```
```yaml
requestId: "{{request_id}}"
notify: true
```
```

#### ## Checklist

- [ ] Resultado persistido.
- [ ] Transação confirmada.
- [ ] Evento correlacionado pelo `jobId`.
- [ ] Evidence Record produzido.

#### Pontuação e consistência

Use ponto final em instruções completas quando isso melhorar a leitura e reduzir a fusão entre itens. Não existe evidência de que terminar toda linha mecanicamente com ponto garanta maior obediência. Consistência semântica, exemplos, contratos e evals possuem efeito mais verificável.

## 7.5 Orçamento de contexto

O objetivo não é usar o menor número absoluto de tokens. É usar contexto suficiente, relevante e verificável.

Uma forma simples de acompanhar o custo é:

$$C_{contexto} = C_{hot} + C_{recuperado} + C_{historico} + C_{ferramentas} + C_{saida}$$

Meça também:

- taxa de recuperação correta;
- precisão da aplicação;
- conteúdo irrelevante carregado;
- latência até a primeira ação útil;
- custo por tarefa concluída;
- retrabalho após avaliação humana.



### **Menos tokens não é automaticamente melhor**

Remover contexto crítico pode reduzir custo por chamada e aumentar erro, repetição e custo total. A unidade econômica correta é a tarefa concluída com qualidade aceitável.

## 7.6 Ciclo de vida dos artefatos

propor -> revisar -> aprovar -> publicar -> observar -> avaliar -> ajustar -> arquivar

Cada mudança relevante deve responder:

- qual problema observável motivou a alteração;
- qual artefato é a fonte de autoridade;
- quais evals precisam mudar;
- qual evidência prova melhora;
- como reverter se houver regressão.

## 7.7 Refatoração orientada por recuperabilidade

Depois de uma reorganização:

1. extraia conceitos-chave e sinônimos dos artefatos;
2. gere consultas positivas, negativas e ambíguas;
3. registre termos usados e conteúdos recuperados;
4. avalie recuperação lexical, semântica e contextual;
5. avalie aplicação no resultado;
6. compare com a versão anterior;
7. ajuste apenas quando houver evidência de problema.

Essa abordagem permite auditoria automática e leitura manual dos arquivos gerados.

## 7.8 Pequeno loop de implementação e testes

Limitar ciclos automáticos reduz repetição improdutiva, mas o número cinco não é uma propriedade universal.

1. Selecionar uma fatia pequena do contrato.
2. Implementar a menor mudança coerente.
3. Executar testes focados e evals relacionados.
4. Registrar evidência e diagnosticar a falha.
5. Repetir no máximo cinco vezes.
6. Se persistir, parar com hipótese, evidências e pergunta objetiva.

Pare antes do limite quando:

- a próxima ação exigir credencial, autoridade ou decisão externa;
- a mesma falha repetir sem nova informação;
- o contrato estiver contraditório;
- a correção ampliar materialmente o escopo;
- o teste não conseguir distinguir sucesso de falso positivo.

## 7.9 Adoção progressiva

A adoção pode avançar em três fases:

- **Fase 1 - piloto:** escolher um domínio, um agent card curto, dois playbooks frequentes, uma skill determinística, uma rule crítica, um contract, uma suíte pequena de evals e um formato de Evidence Record.
- **Fase 2 - comparação:** medir sucesso e retrabalho, comparar custo por tarefa concluída, inspecionar falsos positivos, entrevistar operadores e corrigir nomenclatura ou navegação.
- **Fase 3 - expansão controlada:** definir curadoria, automatizar validações estruturais, versionar contracts e evals, publicar catálogo e acompanhar drift ou conteúdo órfão.

## 7.10 Antipadrões recorrentes

| Antipadrão                         | Por que falha                                          | Correção                                     |
|------------------------------------|--------------------------------------------------------|----------------------------------------------|
| Agent com centenas de linhas       | Dilui prioridade e mistura funções.                    | Manter identidade, invariantes e roteamento. |
| Tudo vira skill                    | Confunde conteúdo consultivo com capacidade ativável.  | Classificar pela função operacional.         |
| Pastas sem política de recuperação | Organização humana não garante descoberta pelo agente. | Criar navegação, metadados e evals.          |
| HOT/WARM/COLD como importância     | Temperatura vira julgamento subjetivo.                 | Definir pela política de carga.              |

| Antipadrão                        | Por que falha                        | Correção                                   |
|-----------------------------------|--------------------------------------|--------------------------------------------|
| SignalR como verdade              | Desconexão perde o estado percebido. | Persistência durável + consulta por API.   |
| Avaliar apenas presença de termos | Premia recuperação superficial.      | Avaliar semântica, aplicação e resultado.  |
| Evidência como texto persuasivo   | Dificulta auditoria e comparação.    | Registrar fatos, IDs, testes e resultados. |

## 7.11 Critérios de sucesso

A estrutura é útil quando produz efeitos observáveis:

- operadores encontram o artefato correto mais rapidamente;
- agentes recuperam menos conteúdo irrelevante;
- obrigações críticas aparecem nos cenários adequados;
- respostas e implementações respeitam os contratos;
- falhas deixam evidências diagnosticáveis;
- custo total por tarefa concluída permanece sustentável;
- a equipe consegue evoluir a base sem depender de uma única pessoa.

## Capítulo 8

# Parte VI - Templates práticos

---

### 8.1 Template de Agent Card

```

id: AGENT-<DOMINIO>-<NÚMERO>
name: <Nome legível do agente>
type: agent
description: <quando este agente deve coordenar a tarefa>
temperature: hot
owner: <time>

```

```
<nome-do-agente>
```

```
Missão
```

```
<resultado principal>
```

```
Invariantes
```

- DEVE ...
- NÃO DEVE ...

```
Roteamento
```

- SE <condição>, USE <playbook ou skill>.

```
Stop conditions
```

- PARE E PERGUNTE quando ...

```
Evidência mínima
```

- REGISTRE ...

### 8.2 Template mínimo de Skill

```

name: <nome-kebab-case-igual-ao-diretorio>
```

```
description: >
 <o que faz, quando usar e termos suficientes para descoberta>
metadata:
 display-name: <Nome Legível>
 owner: <time>
 version: "0.1.0"

<Nome Legível>
```

```
Quando usar
Quando não usar
Entradas e pré-condições
Fluxo principal
Guardrails e stop conditions
Recursos empacotados
Saída e Definition of Done
Evidência e checklist final
```

Comece pelo template mínimo. Acrescente catálogo progressivo, receitas, perguntas ou avisos somente quando os evals demonstrarem necessidade.

## 8.3 Template de Playbook

```

id: PLAYBOOK-<DOMINIO>-<NÚMERO>
name: <Nome legível do playbook>
type: playbook
description: <quando este procedimento coordenado deve ser ativado>
temperature: warm
triggers: []
owner: <time>
status: draft
version: 0.1.0

<nome-do-playbook>
```

```
Quando usar
Pré-condições
Entradas
Procedimento
Decisões condicionais
Fallbacks
Stop conditions
Saídas
```

## Evidências  
## Artefatos relacionados

## 8.4 Template de Rule

```

id: RULE-<DOMINIO>-<NÚMERO>
name: <Nome legível da rule>
type: rule
description: <comportamento obrigatório que esta regra protege>
temperature: warm
authority: normative
owner: <time>
status: draft
version: 0.1.0

```

# RULE-<DOMINIO>-<NÚMERO>

## Obrigação  
## Proibição  
## Fallback  
## Evidência  
## Aplicabilidade  
## Proprietário e vigência

## 8.5 Template de Contract

```
id: CONTRACT-<DOMINIO>-<NÚMERO>
name: <Nome legível da operação>
operation: <identificador-da-operacao>
input:
 required: []
output:
 success: {}
preconditions: []
postconditions: []
forbidden: []
observability:
 correlationFields: []
```

## 8.6 Template de Eval

```
id: EVAL-<DOMINIO>-<NÚMERO>
name: <Nome legível do eval>
```

```
query: <pergunta ou tarefa>
fixtures: []
expected:
 must_retrieve: []
 must_apply: []
 must_not_retrieve: []
 must_not_claim: []
grading:
 method: deterministic-and-judge
 threshold: 0.85
```

## 8.7 Template de Evidence Record

```
evidenceId: EV-<ANO>-<NÚMERO>
requestId: REQ-<ANO>-<NÚMERO>
artifactsRetrieved: []
contractsApplied: []
decisions: []
toolExecutions: []
tests: []
observedResults: []
residualRisks: []
timestamp: <ISO-8601>
```

## 8.8 Checklist do curador

### 8.8.1 Estrutura

- ☐ Cada artefato possui uma função principal reconhecível.
- ☐ Conteúdo normativo não está escondido em exemplo.
- ☐ O agent card continua curto e estável.
- ☐ Playbooks coordenam; skills executam capacidades delimitadas.

### 8.8.2 Recuperação

- ☐ Títulos e descrições usam a linguagem real dos usuários.
- ☐ Sinônimos relevantes aparecem nos metadados ou evals.
- ☐ Consultas negativas evitam recuperação excessiva.
- ☐ O conteúdo recuperado é registrado para auditoria.

### 8.8.3 Aplicação

- ☐ Evals verificam uso correto, não apenas localização.
- ☐ Contracts possuem resultados observáveis.
- ☐ Rules possuem obrigação, proibição, fallback e evidência.
- ☐ A execução deixa Evidence Records suficientes.

#### **8.8.4 Operação**

- ☐ Proprietário, versão e revisão estão definidos.
- ☐ Conflitos possuem política de resolução.
- ☐ Métricas consideram qualidade, latência e custo total.
- ☐ Existe caminho de rollback para mudanças relevantes.

## Capítulo 9

# Glossário

---

Termo	Definição operacional
Agent	Coordenador com missão, invariantes, roteamento e condições de parada.
Agent Card	Conteúdo curto carregado como orientação principal do agente.
Playbook	Procedimento coordenado para uma classe de situações.
Skill	Capacidade ativável, delimitada e verificável.
Knowledge	Conteúdo consultivo que sustenta entendimento e decisão.
Rule	Obrigação, proibição ou fallback com força normativa definida.
Contract	Fronteira verificável de entradas, saídas e condições.
Eval	Caso de avaliação com critérios observáveis.
Evidence Record	Registro estruturado que conecta contexto, decisão, execução e resultado.
Frontmatter	Metadados YAML no topo do artefato, usados por mecanismos de descoberta, governança ou roteamento que os interpretem.
id	Identificador técnico estável para referência e auditoria.
name	Nome de catálogo ou leitura; em Agent Skills, possui regras formais próprias.
HOT	Conteúdo mínimo carregado por padrão.
WARM	Conteúdo recuperado por intenção ou condição frequente.
COLD	Conteúdo profundo ou raro, carregado sob demanda explícita.

Termo	Definição operacional
Progressive disclosure	Entrega gradual de detalhe conforme a necessidade.
NAVIGATION	Conteúdo que aponta caminhos e relações.
LOOKUP	Conteúdo recuperado por consulta direcionada.
DEEP DIVE	Conteúdo de aprofundamento especializado.
NORMATIVE	Conteúdo obrigatório dentro da aplicabilidade declarada.
ADVISORY	Recomendação que admite julgamento contextual.
ILLUSTRATIVE	Exemplo sem força normativa própria.
Recuperabilidade	Capacidade de encontrar o conteúdo correto para uma necessidade.
Idempotência	Propriedade que evita efeitos lógicos duplicados em repetições equivalentes.
SignalR	Tecnologia de comunicação em tempo real entre servidor e clientes .NET.
Fonte de verdade	Estado autoritativo usado para resolver divergências.

## Capítulo 10

# Conclusão

---

Engenharia de contexto não é produzir mais arquivos. É construir um sistema no qual orientação, capacidade, conhecimento, obrigação, contrato, avaliação e evidência tenham funções distintas e conexões verificáveis.

A oficina ensinou o modelo mental sem exigir jargão técnico. A transição mostrou que HOT/WARM/COLD é uma política de carregamento, não uma escala de importância. O backend demonstrou que a separação só se torna confiável quando encontra contracts, rules, evals e Evidence Records.

O objetivo final não é uma árvore de pastas elegante. É uma equipe — humana e assistida por IA — capaz de encontrar o contexto certo, executar dentro dos limites corretos e provar o resultado com custo sustentável.

Para colocar o método em prática, preserve sete decisões:

1. **Comece pela responsabilidade.** Não escolha a pasta antes de entender a função do conteúdo.
2. **Mantenha a orientação central pequena e decisiva.** Papel, invariantes, roteamento, parada e conclusão merecem prioridade.
3. **Separe coordenação de capacidade.** Playbooks organizam jornadas; skills executam atividades delimitadas.
4. **Trate temperatura como carregamento.** HOT, WARM e COLD não substituem autoridade, tipo ou forma de acesso.
5. **Formalize o que precisa ser verificado.** Rules e contracts precisam produzir condições observáveis.
6. **Teste recuperação e aplicação.** Encontrar a fonte correta é necessário, mas não suficiente.
7. **Registre fatos auditáveis.** Evidence Records devem conectar fontes, ações, testes, resultados e riscos.

Comece pequeno, meça o comportamento real e expanda apenas o que demonstrar utilidade. A organização mais valiosa é aquela que permanece compreensível para pessoas, recuperável por agentes e verificável pela engenharia.

## Capítulo 11

# Referências e leituras recomendadas

---

### 11.1 Especificações e guias

- Agent Skills. **Overview**. <https://agentskills.io/home>.
- Agent Skills. **Specification**. <https://agentskills.io/specification>.
- Agent Skills. **How to add skills support to your agent**. <https://agentskills.io/client-implementation/adding-skills-support>.
- Agent Skills. **Optimizing skill descriptions**. <https://agentskills.io/skill-creation/optimizing-descriptions>.
- Model Context Protocol. **Tools**. <https://modelcontextprotocol.io/specification/2026-07-28/server/tools>.
- Model Context Protocol. **Resources**. <https://modelcontextprotocol.io/specification/2026-07-28/server/resources>.
- Anthropic. **Effective context engineering for AI agents**. <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>.
- Anthropic. **Prompting best practices**. <https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/claude-prompting-best-practices>. Acesso em 3 ago. 2026.

### 11.2 Fundamentos de aprendizagem e contexto

- Gentner, D.; Loewenstein, J.; Thompson, L. **Learning and Transfer: A General Role for Analogical Encoding**. Journal of Educational Psychology, 2003. <https://groups.psych.northwestern.edu/gentner/papers/GentnerLoewensteinThompson03.pdf>.
- Liu, N. F. et al. **Lost in the Middle: How Language Models Use Long Contexts**. 2023. <https://arxiv.org/abs/2307.03172>.
- Zeng, Z. et al. **LLMBar: An Open-Source Benchmark for Instruction-Following**. 2023. <https://arxiv.org/abs/2310.07641>.

### 11.3 Tecnologias do estudo de caso

- Microsoft. **ASP.NET Core SignalR overview**. <https://learn.microsoft.com/aspnet/core/signalr/introduction>.

- Microsoft. **Background tasks with hosted services in ASP.NET Core.** <https://learn.microsoft.com/aspnet/core/fundamentals/host/hosted-services>.
- Microsoft. **Azure Service Bus messaging overview.** <https://learn.microsoft.com/azure/service-bus-messaging/service-bus-messaging-overview>.

**i Nota sobre autoria e referências**

Este material organiza conceitos públicos, práticas de engenharia e uma proposta editorial própria. As referências permitem aprofundamento e atribuem as bases utilizadas; os exemplos, templates e decisões de organização foram adaptados ao objetivo didático deste E-book.